Functions

JS functions are effective tools for structuring & organising the code. they are basically are blocks of reuseable codes designeed to perform a specific task.

Functions helps make code modular, clean, & easy to maintain

- **Function Declaration (Name fun.)**

A block of reuseable code that is simply called by its name. They support flexibility, Readability the code organisation.

Function can be defined by using the "function" keyword. & its followed by the name of fun. arguments & the body of functions.

Syntax -

```
function fun-name(para.){
    // code
}
```

Ex-

```
function greet (name) {
    console.log ("Hello" + name);
}

greet ("Saurabh"); // Hello Saurabh
```

Key points -

- declared using the "function" keyword
- Can be called before it is defined (because of hoisting)
- stored in memory at the start of the program execution.

- Function Expression -

A function expression in JS is a way to define a fun & assign it to a variable.

- It can be anonymous or named, & is not hoisted → meaning - it can't be called it is defined.

Syntax →

```
const fun-name =
        function (parameters) {
            // code;
        };
```

Ex -

```
const add = function(a, b) {
    return a+b;
}
console.log(add(5, 3)); //8
```

Key points -

- function is stored in a variable.
- Not hoisted (unlike fun. declaration)
- Commonly used in callbacks or when passing fun. as arguments.

- Anonymous function -

An Anonymous fun. is a fun. without a name. It is often used when a fun. is needed temporarily, for ex- as a callback or inside another fun.

Syntax -

```
function () {
    // code
}
```

Ex -

```
setTimeout (function () {
    console.log ("Hello After 2 seconds");
}, 2000);
```

Key points -

- Can't be called directly (must be assigned or passed)
- ★ Commonly used in fun. expressions & callbacks.
- Introduced mainly for short, one time use fun.

• Arrow functions (ES6 features)

Arrow fun. provide a shorter syntax for writing fun.

Syntax -

```
const fun-name = (parameters) => {
    //code ;
}
```

Ex -

```
//Single line
const square = (x) => { return x*x };
```

अगर यह { } parenthesis नहीं रहे हों तो, Return keyword नहीं mandatory होता।

↳ // multiple lines

```
const greet = (name) => {
    console.log("Hello" + name);
};
```

Key Points -

- don't need the fun. keywords.
- automatically return the value (if single-line)
- don't haved their own this, arguments, or super.

- Callback functions

A callback is a fun. passed as an argument to another fun. and executed later.

Syntax -

```
function mainFunction (callback) {
    // some code
    callback(); // executes the
                   callback
}
```

Example -

```
function sayHello(){
    console.log("Hello!");
}
function greetUser(callback){
    console.log("Welcome user!");
    callback();
}
getUser(sayHello);
```

Key Points -

- Common in asynchronous programming (eg. setTimeout, API calls)
- help execute code after something else has finished.

Example of Callback with setTimeout.

```
setTimeout(()=>{
    console.log("this runs after 2 seconds.");
}, 2000);
```

Here, the arrow function acts as a callback for setTimeout.